ARTIFICIAL INTELLIGENCE

Game Playing

Chapter 5 · Sections 1–5

- Minimax Algorithm
- α–β Pruning
- Resource Limits
- Games of Chance

Lecture Outline

01
Games as Search Problems
Adversarial planning vs. classical search

02
Types of Games
Deterministic vs. chance · Perfect vs. imperfect info

03
Minimax Algorithm
Perfect play in deterministic, perfect-info games

04
α–β Pruning
Eliminating irrelevant branches to search deeper

05
Resource Limits & Evaluation
Cutoff search + heuristic position scoring

06
Games of Chance
Expectiminimax for nondeterministic games

INTRODUCTION

Games vs. Search Problems

Classical Search

Find a fixed sequence of moves that leads to the goal. The environment cooperates.

Adversarial Games

An opponent actively blocks you. Solution must be a contingency plan covering every possible opponent response.

Key Challenges

Time limits
100 sec → ~10^6 nodes. Can't search to leaves.

Contingency plans
Opponent's moves are unpredictable — must plan for all.

Approximation required
Evaluate positions heuristically, not exactly.

Pruning essential
Skip branches that can't affect the result.

Von Neumann (1944) formalized minimax · Shannon (1950) proposed evaluation functions · McCarthy (1956) discovered α–β pruning

TYPES OF GAMES

Classifying Games: A 2×2 Framework

	Deterministic	Chance
Perfect Information	Chess · Checkers Go · Othello ← This chapter's focus	Backgammon Monopoly 🎲 Dice determine legal moves
Imperfect Information	*(Rare)*	Bridge · Poker Scrabble 🃏 Hidden cards + dice

Algorithm complexity increases → from top-left to bottom-right

MINIMAX

Minimax: The Core Idea

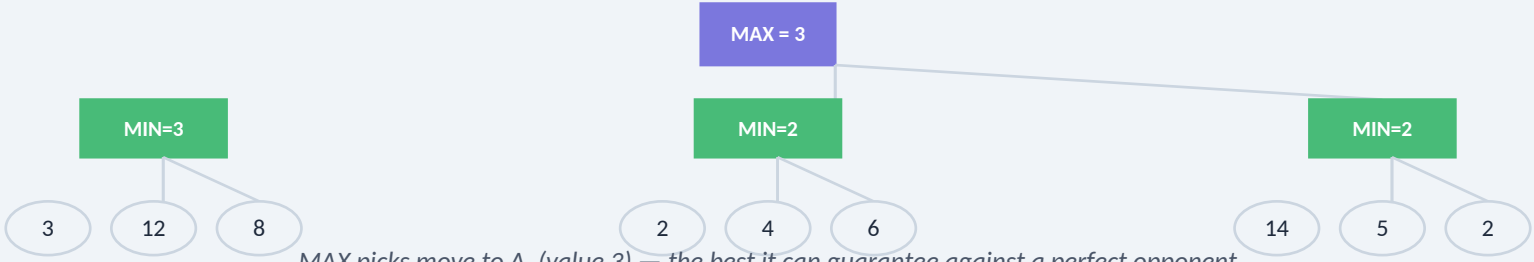
"Choose the move that leads to the position with the highest *minimax value*" — best achievable payoff against best play

MAX (You)

Wants to MAXIMIZE the outcome.
Chooses the child with the highest value.

MIN (Opponent)

Wants to MINIMIZE the outcome.
Chooses the child with the lowest value.



MAX picks move to A_1 (value 3) — the best it can guarantee against a perfect opponent

MINIMAX

The Minimax Algorithm

```
function MINIMAX-DECISION(game)
  for each op in OPERATORS[game]:
    VALUE[op] ← MINIMAX-VALUE(APPLY(op, game), game)
  return op with highest VALUE[op]
```

```
function MINIMAX-VALUE(state, game)
  if TERMINAL-TEST[game](state):
    return UTILITY[game](state)  ← leaf node
  else if MAX to move:
    return MAX of MINIMAX-VALUE(successors)
  else:
    return MIN of MINIMAX-VALUE(successors)
```

Recursive

Calls itself on every successor state until terminal nodes are reached.

Two roles

Alternates between MAX (picks highest) and MIN (picks lowest) at each level.

Complete & Optimal

Finds the best move if the tree is finite. Optimal against a perfect opponent.

PROPERTIES

Properties of Minimax

Complete?

YES

If the game tree is finite (chess has termination rules).

Optimal?

YES

Against a perfect opponent — guaranteed best outcome.

Time complexity

$O(b^m)$

Exponential in depth. For chess: 35^{100} — totally infeasible.

Space complexity

$O(bm)$

Linear — depth-first, only one path stored in memory at once.

Chess: $b \approx 35$, $m \approx 100$ → 35^{100} nodes → exact solution completely infeasible → we need cutoffs + evaluation

RESOURCE LIMITS

Cutting Off Search: Evaluation Functions

The Problem

100 seconds/move × 10^4 nodes/sec = only 10^6 nodes per move. Game tree is orders of magnitude larger.

The Solution

Replace TERMINAL with CUTOFF (depth limit) + Replace UTILITY with EVAL (heuristic)

EVAL(s) for Chess:

$$\mathrm{EVAL}(s) = \Sigma\, w_i \times f_i(s)$$

e.g. f_1 = #white queens − #black queens ($w_1 = 9$)

Standard Chess Piece Values

Piece	Symbol	Weight
Queen	♕	9
Rook	♖	5
Bishop	♗	3
Knight	♘	3

Quiescence search: extend cutoff at 'unstable' positions (captures, checks) to avoid horizon effects.

Search depth vs. playing strength:

ALPHA-BETA PRUNING

α–β Pruning: Search Smarter

Prune branches that cannot possibly influence the final decision — identical result to full Minimax, less work.

α (alpha)

Best value MAX has found along the current path.
MAX will never choose below α.

β (beta)

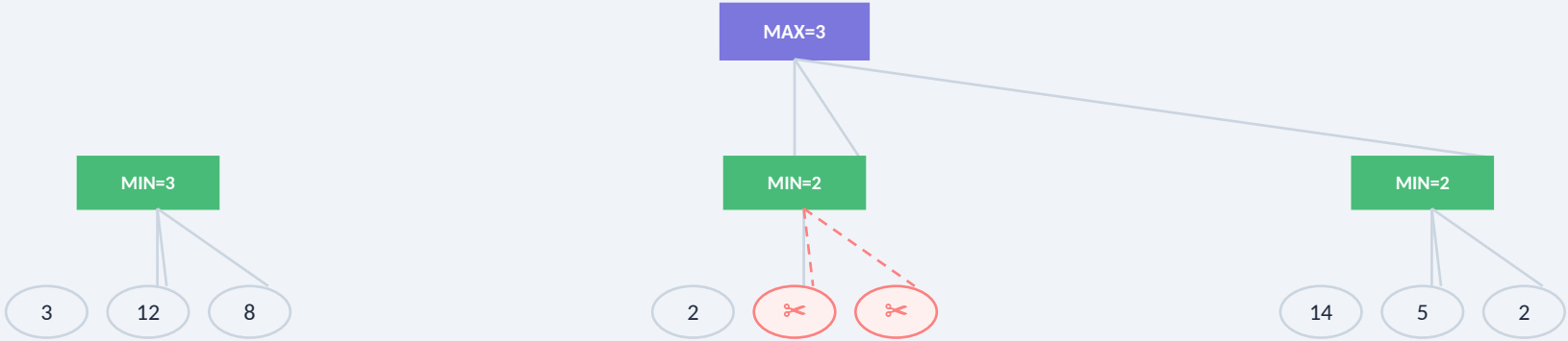
Best value MIN has found along the current path.
MIN will never choose above β.

Pruning rule: At MIN node, if current value v ≤ α → PRUNE. MAX already has something better up the tree.

Condition	Time Complexity	Chess Depth
Worst case (no ordering)	$O(b^m)$	~4 ply
Random move ordering	$O(b^{3m/4})$	~6 ply
Best case (perfect ordering)	**O(b^(m/2))**	**~8 ply ✓**

ALPHA-BETA PRUNING

α–β Pruning: Step-by-Step Example



How pruning unfolds:

1. Evaluate MIN_1 children: {3, 12, 8} → MIN_1 = 3. MAX updates α = 3.
2. Evaluate MIN_2 first child: 2. Since 2 ≤ α (3), MAX will never choose this branch.
3. PRUNE ✂ Skip remaining children of MIN_2 — they can't help MAX. 2 nodes saved.
4. Evaluate MIN_3: {14, 5, 2} → MIN_3 = 2. MAX = max(3, 2, 2) = 3. Same as full minimax!

Result: explored 7 of 9 nodes — same answer as full minimax. With perfect ordering this saving scales dramatically.

MAIN STEPS (Alpha–Beta Pruning Procedure)

Step 1: Start from Root

- Assign:
  - **$\alpha = -\infty$**
  - **$\beta = +\infty$**
- Identify:
  - Root is **MAX** (usually)

Step 2: Traverse Depth-First (Left → Right)

- Always explore **one branch completely**
- Go down to **leaf nodes first**

Step 3: Apply Minimax Rules

- **MAX node:**
  - Choose **maximum**
  - Update:

$$\alpha = \max(\alpha, \text{child value})$$

- **MIN node:**
  - Choose **minimum**
  - Update: $\beta = \min(\beta, \text{child value})$

Step 4: Check Pruning Condition

➢ After each child evaluation:

$$\alpha \geq \beta$$

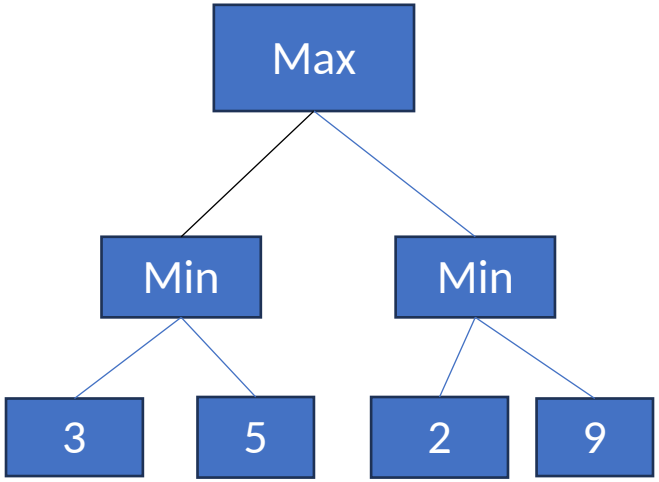
- If TRUE → **STOP exploring remaining children**
- If FALSE → **continue**

MAIN STEPS (Alpha–Beta Pruning Procedure)

Step 5: Prune Remaining Nodes
- Do **NOT evaluate further children**
- Skip entire remaining subtree from that node

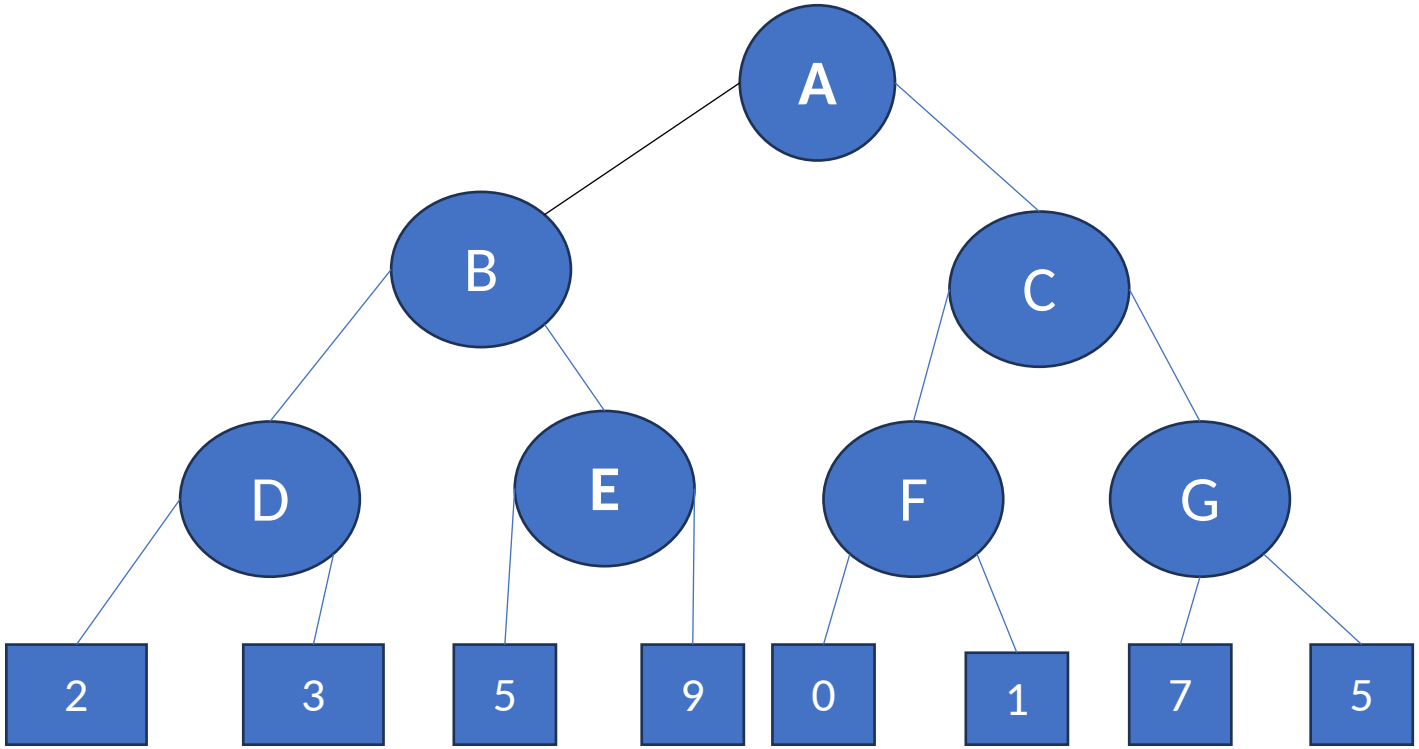
Step 6: Propagate Values Upward
- Pass computed value to parent
- Continue same process

Step 7: Final Decision at Root
- Root (MAX) chooses **maximum value**
- That is the **optimal move**



Try with the following terminal:

Try Alpha-Beta Pruning with this Example:



IN PRACTICE

Deterministic Games in Practice

Checkers ♖

Chinook (1994)

- Defeated world champion Marion Tinsley
- Endgame database: 443,748,401,247 positions
- Used an endgame DB for all ≤8-piece positions
- Perfect play proven — game is solved!

Chess ♕

Deep Blue (1997)

- Defeated Kasparov in 6-game match
- Searched 200M positions/second
- Very sophisticated evaluation function
- Extended some lines to 40 ply

Othello ●

Too good for humans

- Human champions refuse to compete
- Positional features (corners, stable discs)
- Evaluation functions extremely refined
- Computers dominate completely

Go ⬡

Broke these methods

- Branching factor $b > 300$
- Traditional α–β fails completely
- Pattern knowledge bases for plausible moves
- Only AlphaGo (2016, deep RL) succeeded

GAMES OF CHANCE

Nondeterministic Games: Expectiminimax

When dice, cards, or other random events are involved, we add a third node type:

▲ MAX node

You choose — pick action with highest expected value.

▼ MIN node

Opponent chooses — pick action with lowest expected value.

◆ CHANCE node

Random event — take weighted average over all outcomes.

```
If state is CHANCE node:  return  Σ  P(outcome) × EXPECTIMINIMAX(outcome)
```

Example: Backgammon

21	20	420	depth-2
possible dice rolls	legal moves / roll	effective branch factor	TDGAMMON search depth

⚠ Unlike deterministic games: exact EVAL values matter! Expected value calculation requires proportional scale.

CHAPTER 5 SUMMARY

Key Takeaways

1. Games require contingency plans — not fixed move sequences. The opponent's choices must be anticipated.
2. Minimax gives optimal play in deterministic, perfect-info games but is computationally infeasible for real games.
3. α–β pruning delivers the same result as Minimax while skipping irrelevant branches — doubling search depth.
4. Resource limits force us to cut off search and use evaluation functions to estimate position quality.
5. In chance games, Expectiminimax handles random nodes; exact EVAL scale matters (unlike deterministic games).

Games are to AI as Grand Prix racing is to automobile design — a rich, challenging test domain.